

Codex Implementation Specification

A complete, ordered engineering contract for a secure multi-clinic website builder, appointment platform, patient CRM, and clinic operations MVP.

Document	Value
Version	2.0 - corrected implementation edition
Status	Build-ready baseline
Primary consumer	Codex and the engineering team
Release target	Commercial MVP
Architecture	Next.js + FastAPI modular monolith + PostgreSQL
Date	September 2026

Authority: This document replaces ambiguous implementation details in the earlier CRM stack draft. If code, tickets, or informal notes conflict with this specification, Codex must stop, identify the conflict, and request a documented decision before changing the contract.

Simple on the surface. Disciplined underneath.

Contents

Contents	2
1. How Codex Must Use This Specification	5
1.1 Instruction precedence	5
1.2 Required working loop	5
1.3 Non-negotiable stop conditions	5
2. Product Definition and Boundaries	5
2.1 MVP outcomes	5
2.2 Included	5
2.3 Explicit non-goals	6
3. Requirement Language and Traceability	6
3.1 Required traceability record	6
4. Final Technology Decisions	6
5. System Architecture and Trust Boundaries	7
5.1 Domain boundaries	7
5.2 Domain and cookie topology	7
6. Repository Contract	7
6.1 Backend module contract	7
6.2 Shared contracts	8
7. Canonical Data Conventions	8
7.1 Required base fields	8
8. Complete Database Inventory	8
8.1 Core entity requirements	9
9. Database Integrity and Tenant Isolation	9
9.1 Mandatory same-tenant relationships	9
9.2 Mandatory PostgreSQL RLS	9
9.3 Service and migration roles	9
9.4 Automated isolation matrix	9
10. Appointment Engine - Exact Concurrency Contract	10
10.1 Time calculation	10
10.2 Database overlap prevention	10
10.3 Booking transaction	10
11. Appointment State Machine	10
11.1 Transition API	10
12. Availability, Hours, Holidays, and Resources	11
13. Authentication and Account Recovery	11
13.1 Password and session contract	11
13.2 MFA	11
13.3 No-email/SMS onboarding workflow	11
14. Authorization Contract	12

14.1 Permission catalog	12
14.2 Object scope	12
15. API Standards and Error Contract	12
15.1 General rules	12
15.2 Response envelopes	12
16. Complete API Surface	13
16.1 Public booking-management token	13
17. Patient CRM, Duplicates, Notes, and Consent	13
17.1 Duplicate control	13
17.2 Note visibility	14
17.3 Consent and privacy requests	14
18. Private Files and Public Media	14
18.1 Upload pipeline	14
18.2 Default limits	14
19. Website Builder and Publishing	14
19.1 Domain verification	14
19.2 Content safety	15
20. Dashboard, Queue, Follow-ups, and Notifications	15
20.1 Dashboard source of truth	15
20.2 Queue	15
20.3 Follow-ups	15
20.4 Browser notifications	15
21. Security and Privacy Baseline	15
21.1 Security headers	15
21.2 Encryption and secrets	15
22. Audit, Logging, and Observability	16
22.1 Audit events	16
22.2 Required telemetry	16
22.3 Health endpoints	16
23. Performance, Accessibility, and Reliability Targets	16
23.1 Load profile for pilot gate	16
23.2 Accessibility acceptance	17
24. Backups, Retention, and Disaster Recovery	17
24.1 Disaster runbook order	17
25. CI/CD and Environment Contract	17
25.1 Pull request gate	17
25.2 Release gate	17
25.3 Environment variables	17
26. Ordered Codex Build Plan	18
27. Codex Task Template	18
27.1 Codex completion report	18

27.2 Prohibited shortcuts	18
28. Testing Contract	19
28.1 Test-data rule	19
29. Feature Definition of Done	19
30. Production Launch Gate	19
31. Resolved Gaps and Decision Register	20
32. Final Acceptance Statement	20

1. How Codex Must Use This Specification

This is an execution contract, not a product essay. Codex must implement in the ordered phases in Section 26 and may not silently invent security, data, workflow, or infrastructure behavior.

1.1 Instruction precedence

Priority	Source	Required behavior
1	Explicit user instruction	Follow it unless it creates a security or data-integrity conflict; surface conflicts.
2	This specification	Treat decisions marked MUST as binding.
3	Repository AGENTS.md and ADRs	Apply repository-local rules that do not contradict higher authority.
4	Existing code	Preserve working behavior only when it matches the approved contract.
5	Framework defaults	Use only when the contract is silent and record the decision.

1.2 Required working loop

- Read the current phase, its dependencies, acceptance tests, and Definition of Done before editing.
- Inspect the repository and migrations; do not assume a blank project.
- Create or update a short implementation plan tied to requirement IDs.
- Implement the smallest vertical slice that can be tested end to end.
- Add migration, backend authorization, audit behavior, tests, frontend states, and documentation together.
- Run lint, type checks, unit/API tests, migration checks, and relevant Playwright flows.
- Report changed files, commands run, test results, remaining risks, and any deliberate deviation.

1.3 Non-negotiable stop conditions

- Tenant scope cannot be proven for a clinic-owned query.
- A database migration could destroy or irreversibly transform production data.
- A requested change would expose patient information, secrets, or signed URLs.
- A product decision changes booking semantics, privacy obligations, or role permissions.
- A production credential, legal compliance claim, or external integration is required but not approved.

Codex may create local test fixtures and reversible development migrations. It must never place real patient data in fixtures, logs, screenshots, prompts, or source control.

2. Product Definition and Boundaries

A multi-tenant healthcare platform enabling independent doctors and small clinics to publish premium websites, accept appointment requests, manage schedules, operate a live reception queue, maintain a basic patient CRM, and manage follow-ups from one dashboard.

2.1 MVP outcomes

- A non-technical clinic publishes a complete responsive website in under 30 minutes.
- A mobile patient submits a booking request in under three minutes.
- Clinic staff manage the complete appointment lifecycle without spreadsheets.
- No authenticated user can access another clinic's data through UI, API, storage, exports, or background work.
- The product supports one doctor or multiple doctors and branches without architecture replacement.

2.2 Included

Clinic onboarding; three controlled website templates; branding and section editor; doctors, branches and services; public booking; availability; approvals; calendar; reception queue; patient CRM; notes, tags and private documents; manual follow-ups; internal and browser notifications; RBAC; audit; plans and limits; super-admin; backups, monitoring and support access.

2.3 Explicit non-goals

No payment processing, WhatsApp, email or SMS sending, insurance claims, AI diagnosis, full EMR, pharmacy, laboratory, native mobile apps, microservices, Kubernetes, Redis, unrestricted website canvas, or unapproved LLM dependency.

Authentication recovery and staff onboarding must work without email/SMS. The approved manual one-time-link workflow is defined in Section 13.

3. Requirement Language and Traceability

MUST means release-blocking. SHOULD means expected unless an ADR records a reason. MAY means optional. Every issue and pull request must reference at least one requirement ID.

Prefix	Domain	Examples
TEN	Tenancy	TEN-001 mandatory tenant context; TEN-004 cross-tenant tests
SEC	Security/privacy	SEC-006 CSRF; SEC-014 log redaction
AUT	Authentication	AUT-003 hashed sessions; AUT-012 manual recovery
RBAC	Authorization	RBAC-002 API enforcement
APT	Appointments	APT-010 exclusion constraint; APT-016 idempotency
CRM	Patient CRM	CRM-008 private note visibility
WEB	Website	WEB-011 atomic publish
OPS	Operations	OPS-005 queue ordering
REL	Reliability	REL-004 restore test
UX	Experience	UX-003 tablet workflows

3.1 Required traceability record

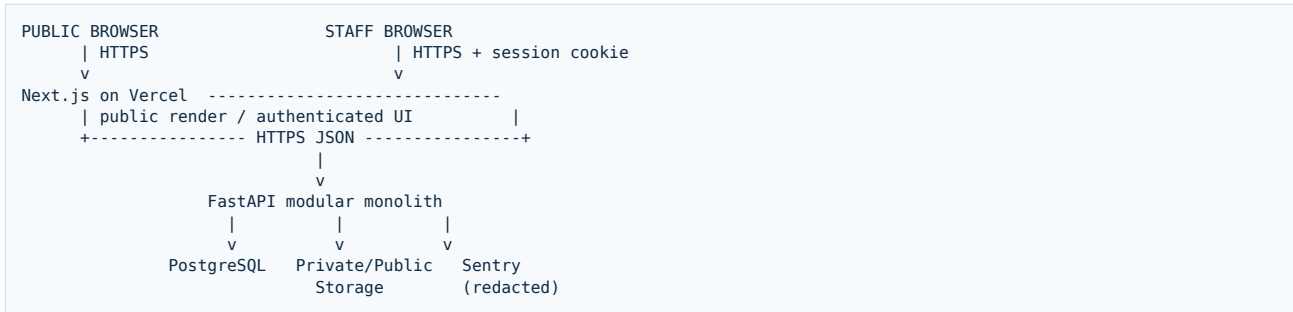
Issue: APT-010 Prevent overlapping doctor appointments
Migration: 2026xxxx_add_appointment_exclusion.py
API: POST /api/v1/public/bookings
Tests: test_concurrent_booking.py, booking-concurrency.spec.ts
Evidence: CI run URL + migration upgrade/downgrade result

4. Final Technology Decisions

Layer	Binding decision	MVP rule
Web	Next.js, React, TypeScript	One application: public sites and authenticated dashboard.
Web hosting	Vercel	Preview deployments allowed; production domain is controlled.
API	FastAPI, Python, Pydantic 2	Versioned JSON API; modular monolith.
API hosting	Railway	Single production provider; Render is an approved contingency, not parallel runtime.
Database	Managed PostgreSQL 16	Primary source of truth; UTC timestamps.
ORM/migrations	SQLAlchemy 2 + Alembic	Explicit repositories/services; reviewed migrations.
Storage	Supabase Storage	Separate public-assets and private-patient buckets.
Sessions	Opaque server-side sessions	Hashed tokens in PostgreSQL; secure cookies.
MFA	TOTP + recovery codes	Mandatory for owners and platform administrators.
Monitoring	Sentry + structured platform logs	Patient data redacted.
Testing	Pytest + Playwright	Unit, API, isolation, concurrency, and critical browser flows.
CI	GitHub Actions	Protected main; staging gate before production.
Local runtime	Docker Compose	Web, API and PostgreSQL-compatible development services.
Cache/jobs	Database-backed jobs only	No Redis until measured load justifies it.

ADR-001: Railway is the default API host to eliminate the earlier Railway-or-Render ambiguity. Changing provider requires an architecture decision record and verified backup/deployment runbooks.

5. System Architecture and Trust Boundaries



5.1 Domain boundaries

Module	Owns	Must not own
identity	users, sessions, MFA, reset/invite tokens	clinic business workflows
tenancy	clinics, branches, plans, feature limits	appointments
catalog	doctors, services, rooms/resources	patient records
scheduling	availability, leave, holidays, slot calculation	website content
appointments	bookings, status transitions, history	passwords
crm	patients, contacts, tags, notes, consent	public website versions
files	metadata, scan state, signed access	raw storage authorization bypass
websites	pages, sections, media, versions, domains	clinical documents
operations	queue, follow-ups, notifications	tenant identity
audit	append-only security and business events	mutable business state
admin	clinic lifecycle and time-limited support access	routine clinical access

5.2 Domain and cookie topology

- Authenticated app: app.platform.example. API: api.platform.example.
- Session cookie Domain=.platform.example; Secure; HttpOnly; SameSite=Lax; Path=/; explicit Max-Age.
- Custom clinic domains serve public content only and never receive dashboard session cookies.
- CORS allowlist contains only approved app origins. Preview deployments use separate non-production sessions.
- All state-changing cookie-authenticated requests require Origin validation and CSRF token header.
- Backend-to-storage calls use service credentials that never reach the browser.

6. Repository Contract

apps/ web/ app/(public)/ components/ api/ app/api/v1/ app/db/ packages/ contracts/ infra/ docker/ docs/ adr/ security/	app/(dashboard)/ features/ lib/ app/modules/ tests/ design-tokens/ github/ api/ runbooks/ database/ requirements/	
---	---	--

6.1 Backend module contract

Each module uses routes.py, schemas.py, models.py, repository.py, service.py, permissions.py, events.py where needed, and tests. Routes validate transport; services enforce business rules; repositories require TenantContext; models define persistence. No route performs unscoped ORM queries.

6.2 Shared contracts

- OpenAPI generated by FastAPI is the canonical API description.
- TypeScript API types are generated from the committed OpenAPI artifact.
- Frontend must not duplicate status enums or permission codes manually.
- Database migrations and enum changes are backward compatible across one deployment window.

7. Canonical Data Conventions

Concern	Contract
Identifiers	UUID v7 preferred; immutable; generated server-side.
Time	timestampz stored in UTC; clinic and branch IANA timezone names.
Money	integer minor units + ISO currency; display only in MVP, never floating point.
Deletion	archive/soft delete operational records; hard delete only through approved privacy workflow.
Concurrency	updated_at plus integer version for optimistic locking where editing collisions matter.
Tenant keys	Every clinic-owned row has clinic_id NOT NULL and indexed.
Uniqueness	Tenant-local values use UNIQUE(clinic_id, value); global uniqueness only where required.
JSON	Use jsonb for versioned flexible content only; validate with Pydantic/schema version.
Search	Normalized phone/email columns; trigram indexes only after measured need.
Audit	Append-only; metadata is redacted and size-bounded.

7.1 Required base fields

```
id uuid PRIMARY KEY
clinic_id uuid NOT NULL REFERENCES clinics(id)
created_at timestampz NOT NULL DEFAULT now()
updated_at timestampz NOT NULL DEFAULT now()
archived_at timestampz NULL
version integer NOT NULL DEFAULT 1
```

8. Complete Database Inventory

The following inventory closes the missing-table gaps. Detailed migrations remain the source of executable truth and must preserve these relationships.

Domain	Required tables
Platform	plans, feature_limits, clinics, clinic_subscriptions, system_announcements
Organization	branches, branch_hours, clinic_holidays, rooms, resources
Identity	users, roles, permissions, role_permissions, user_roles, user_branch_scopes, sessions, mfa_methods, recovery_codes, access_tokens, staff_invitations, password_reset_tokens
Clinical catalog	doctor_profiles, services, doctor_services, branch_doctors, branch_services, doctor_rooms
Scheduling	availability_rules, leave_blocks, blocked_slots, resource_blocks
Appointments	appointments, appointment_history, booking_questions, service_booking_questions, appointment_answers, booking_management_tokens
CRM	patients, patient_contacts, patient_tags, tags, patient_notes, consent_records, patient_merge_events
Files	patient_documents, file_scan_events, website_media
Operations	queue_entries, follow_up_tasks, notifications, browser_push_subscriptions
Websites	websites, website_pages, website_sections, website_versions, domain_verifications
Administration	support_access_sessions, audit_events, export_jobs, privacy_requests, background_jobs

8.1 Core entity requirements

Entity	Minimum binding fields and constraints
clinics	id; name; slug UNIQUE; status; plan; timezone; locale; contact fields; retention_policy_id; timestamps
branches	clinic_id; code; name; timezone; address; phone; status; UNIQUE(clinic_id, code)
users	clinic_id nullable only for platform admins; normalized_email; password_hash; status; failed_login_count; locked_until; last_login_at
doctor_profiles	clinic_id; user_id nullable; public name; specialty; registration; verification; bio; consultation duration; status
services	clinic_id; name; category; descriptions; duration; buffers; price_mode; amount/range/currency; approval_mode; visibility
patients	clinic_id; patient_number; name; normalized phone/email; DOB; gender optional; preferred doctor/branch; status; duplicate_of nullable
appointments	clinic, branch, doctor, service, patient; starts/ends; blocking_range; status; source; reference; idempotency key; policy snapshot; version
patient_notes	clinic; patient; author; note_type; visibility; body; version; no reception access to private_doctor
patient_documents	clinic; patient; uploader; generated storage key; metadata; hash; scan_status; retention class
follow_up_tasks	clinic; patient/appointment; assignee; reason; due_at; priority; status; outcome; completed_at
website_versions	clinic; website; version; schema_version; immutable snapshot; creator; published_at; checksum
audit_events	clinic nullable for platform event; actor; support session; action; entity; request_id; IP hash; metadata; created_at; immutable

9. Database Integrity and Tenant Isolation

9.1 Mandatory same-tenant relationships

Every cross-entity relationship between clinic-owned rows MUST be protected by a composite foreign key. Each referenced table exposes UNIQUE(clinic_id, id). Application checks are additional, not substitutes.

```
ALTER TABLE patients
  ADD CONSTRAINT uq_patients_tenant_id UNIQUE (clinic_id, id);

ALTER TABLE appointments
  ADD CONSTRAINT fk_appointment_patient_tenant
  FOREIGN KEY (clinic_id, patient_id)
  REFERENCES patients (clinic_id, id) ON DELETE RESTRICT;
```

9.2 Mandatory PostgreSQL RLS

RLS is mandatory on patients, appointments, notes, documents, follow-ups, queue entries, notifications, website drafts, and other clinic-owned tables. The API transaction sets app.clinic_id and app.user_id from the validated session; client input can never set them.

```
ALTER TABLE patients ENABLE ROW LEVEL SECURITY;
ALTER TABLE patients FORCE ROW LEVEL SECURITY;
CREATE POLICY patient_tenant_policy ON patients
  USING (clinic_id = current_setting('app.clinic_id', true)::uuid)
  WITH CHECK (clinic_id = current_setting('app.clinic_id', true)::uuid);
```

9.3 Service and migration roles

- Runtime database role is subject to RLS and cannot alter schema.
- Migration role is separate, stored outside application runtime, and used only by controlled release jobs.
- Platform support does not use a universal database bypass. It receives a time-limited audited tenant context.
- Background jobs carry an explicit clinic_id and set the same RLS context before data access.

9.4 Automated isolation matrix

For two clinics, test read, list, search, create with foreign IDs, update, archive, export, signed download, public token use, websocket/browser notification access, background jobs, and support expiry. Every unauthorized operation returns 404 or 403 according to the endpoint contract and records a security event where appropriate.

10. Appointment Engine - Exact Concurrency Contract

10.1 Time calculation

- starts_at is the patient-facing appointment start.
- occupancy_start = starts_at - buffer_before; occupancy_end = starts_at + duration + buffer_after.
- Store the service duration and policy snapshot on the appointment so later service edits do not rewrite history.
- All calculations use branch timezone for rules and UTC for persistence.

10.2 Database overlap prevention

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE appointments ADD COLUMN occupancy tstzrange
    GENERATED ALWAYS AS (tstzrange(occupancy_start, occupancy_end, '[')') STORED;

ALTER TABLE appointments ADD CONSTRAINT no_doctor_overlap
    EXCLUDE USING gist (
        clinic_id WITH =, doctor_id WITH =, occupancy WITH &&
    ) WHERE (blocks_time = true);
```

blocks_time is derived by the service layer from status and the clinic's requested-appointment policy and is database-validated. Equivalent exclusion constraints apply to exclusive rooms/resources when configured.

10.3 Booking transaction

```
BEGIN;
1. Resolve tenant and validate service/doctor/branch relationships.
2. Lock or re-evaluate applicable availability and daily-limit rows.
3. Validate hours, leave, blocks, notice, horizon, capacity and resources.
4. Upsert idempotency record scoped to tenant + endpoint + key.
5. Insert patient or attach duplicate-reviewed existing patient.
6. Insert appointment; exclusion constraint is final conflict authority.
7. Insert initial history and audit event.
COMMIT;
```

- A conflict returns 409 APPOINTMENT_CONFLICT with no internal identifiers.
- Public POST requests require Idempotency-Key; the same key and same body returns the original safe response for 24 hours; a different body returns 409 IDEMPOTENCY_MISMATCH.
- Rescheduling creates a replacement appointment linked by supersedes_appointment_id, then marks the original Rescheduled in the same transaction.
- A booking hold is not used in MVP. A displayed slot is advisory; commit-time validation is authoritative.

11. Appointment State Machine

Current	Allowed next state	Required guard/effect
Requested	Confirmed, Cancelled	Approval permission; cancellation reason when staff rejects.
Confirmed	Arrived, Rescheduled, Cancelled, No-show	Arrival requires clinic day; no-show records actor/time.
Arrived	Waiting, Cancelled	Create or update one active queue entry.
Waiting	In consultation, Cancelled	Doctor is assigned and authorized.
In consultation	Completed	Completion timestamp; optional follow-up.
Completed	None	Administrative correction is a separate audited action.
Cancelled	None	Replacement requires new appointment.
No-show	None	Recovery follow-up may be created.
Rescheduled	None	Points to replacement appointment.

11.1 Transition API

```
POST /api/v1/appointments/{id}/transitions
{
  "to_status": "confirmed",
  "reason_code": null,
  "note": null,
  "expected_version": 4
}
```

Generic PATCH cannot change status, schedule, doctor, branch, or service. Dedicated commands preserve invariants: /transitions, /reschedule, /assign, and /cancel. Stale expected_version returns 409 VERSION_CONFLICT.

12. Availability, Hours, Holidays, and Resources

- branch_hours stores weekday intervals and supports multiple intervals per day.
- clinic_holidays stores full-day or time-range closures, clinic-wide or branch-specific.
- availability_rules store doctor weekly intervals, effective date range, branch, slot cadence, and optional service scope.
- leave_blocks and blocked_slots override recurring availability.
- rooms/resources have capacity=1 in MVP; service requirements identify required resource types.
- Slot generation intersects branch hours, doctor rules, service eligibility, resource availability, booking notice/horizon, and daily limits.
- DST gaps and repeated local times are resolved using IANA timezones; ambiguous times are returned with explicit UTC offsets.

Rule	Default
Slot cadence	15 minutes
Minimum notice	120 minutes
Booking horizon	90 days
Same-day booking	Allowed when minimum notice is satisfied
Requested appointment blocking	Blocks time by default
Daily limit	Unlimited unless doctor/clinic config sets a value
Public availability cache	No application cache in MVP; indexed query with short HTTP cache only if safe

13. Authentication and Account Recovery

13.1 Password and session contract

- Passwords use Argon2id with parameters reviewed annually; never reversible encryption.
- Session token contains at least 256 bits of randomness; only SHA-256/HMAC hash is stored.
- Rotate session after login, MFA completion, privilege change, password reset, and support-context change.
- Idle timeout: 30 minutes for platform admin; 8 hours for clinic staff. Absolute lifetime: 24 hours and 7 days respectively.
- Rate limit by normalized account plus privacy-preserving IP key; generic login response prevents enumeration.

13.2 MFA

Owners and platform admins must enroll TOTP before full access. Store encrypted TOTP secret using an application encryption key separate from the database. Generate ten single-use hashed recovery codes. Enrollment and recovery are audited. Receptionist and doctor MFA may be required by clinic policy.

13.3 No-email/SMS onboarding workflow

- Owner creates a staff invitation. The API returns a one-time HTTPS setup link exactly once.
- The owner delivers the link manually outside the platform. The platform does not claim or perform message delivery.
- The token is hashed, single-use, expires in 24 hours, is bound to clinic and intended normalized email/username, and is invalidated when staff access is deactivated.
- Forgot-password does not send a message. The user contacts the clinic owner; the owner verifies identity outside the system and issues a one-time reset link. Owner recovery requires platform-admin verification and two-person approval in production.
- The login page must state that recovery requires contacting the clinic administrator; it must not pretend an email was sent.

If automated email or SMS is later approved, it is a separately threat-modeled integration. The token model can remain, but delivery status and abuse controls must be added.

14. Authorization Contract

UI visibility is convenience only. Every API command and resource lookup performs backend permission and object-scope checks after tenant resolution.

Role	Core scope	Explicit restriction
Owner	All clinic configuration and records; staff; publish; audit	No platform controls or other tenants
Manager	Operations, staff, catalog, reports; publish if granted	No owner recovery or platform controls
Doctor	Own schedule; assigned/allowed patients; own private notes; follow-ups	No unrelated private notes or staff administration
Receptionist	Appointments, check-in, queue, basic patient data, permitted documents	No private doctor notes or security settings
Website editor	Draft pages, media, services display, SEO	No patients, appointments, documents, or publish by default
Platform admin	Clinic lifecycle, plans, health, explicit support sessions	No routine clinical access

14.1 Permission catalog

clinic.read/update; branch.read/manage; staff.read/manage; doctor.read/manage; service.read/manage; schedule.read/manage; appointment.read/create/manage/approve/cancel/reschedule/check_in; queue.read/manage; patient.read/create/update/archive/export; patient.note.read/write; patient.private_note.read/write; patient.document.read/write/delete; consent.read/manage; followup.read/manage; notification.read/manage; website.read/edit/publish; audit.read; admin.clinic.manage; admin.plan.manage; admin.support.access; admin.analytics.read.

14.2 Object scope

- Doctor access is evaluated using assigned doctor, care-team relationship, explicit clinic policy, and branch scope.
- Reception document access is a distinct permission and may be disabled.
- Support access specifies clinic, requested permissions, reason, approver, expiry of at most 60 minutes, and revocation.
- Sensitive reads, exports, downloads, permission changes, and support actions are audited.

15. API Standards and Error Contract

15.1 General rules

- Base path /api/v1; JSON UTF-8; snake_case fields; RFC 3339 timestamps with offsets.
- OpenAPI descriptions include permission, tenant behavior, idempotency, examples, and stable errors.
- Collections use cursor pagination by default; page pagination is allowed only for stable small admin tables.
- All responses include X-Request-ID; clients may send one but the server validates or replaces it.
- PATCH requires expected_version for contention-sensitive records.

15.2 Response envelopes

```
200
{"data": {...}, "meta": {"request_id": "uuid"}}

Error
{"error": {"code": "APPOINTMENT_CONFLICT",
"message": "The selected time is no longer available.",
"fields": null},
"meta": {"request_id": "uuid"}}
```

HTTP	Use	Examples
400	Normalized input error	INVALID_INPUT, INVALID_TRANSITION
401	Missing/expired authentication	SESSION_EXPIRED, MFA_REQUIRED
403	Known resource but insufficient privilege	FORBIDDEN

HTTP	Use	Examples
404	Absent or outside authorized scope	NOT_FOUND
409	Concurrency/business conflict	VERSION_CONFLICT, APPOINTMENT_CONFLICT
413	Upload too large	FILE_TOO_LARGE
422	Framework validation only if intentionally exposed	Otherwise normalize to 400
429	Rate limit	RATE_LIMITED
500	Generic unexpected failure	INTERNAL_ERROR; no sensitive detail

16. Complete API Surface

Area	Required endpoint groups
Auth	POST login/logout; POST mfa/enroll, mfa/verify, mfa/recover; GET me; POST sessions/revoke; POST manual-reset/consume
Clinics	GET/PATCH clinic; branches CRUD; hours CRUD; holidays CRUD; rooms/resources CRUD
Staff	invitations create/revoke; users list/detail/status; roles/permissions; branch scopes
Catalog	doctors CRUD; services CRUD; doctor/service/branch assignments; preparation content
Scheduling	availability rules CRUD; leave CRUD; blocked time CRUD; public/scoped slot query
Appointments	public create/manage; scoped list/detail; transitions; approve; reject; cancel; reschedule; assign; history
Operations	dashboard summary; queue list/check-in/reorder/start; notifications list/read; activity
Patients	list/search/detail/create/update/archive; duplicate candidates; merge; history; contacts; tags
Notes/consent	notes create/update; private visibility; consent record list/create/revoke
Files	upload-init or streamed upload; scan status; authorized download; delete/archive
Follow-ups	list/create/update/assign/complete; due counters
Websites	site/page/section/media CRUD; preview token; publish; versions; rollback; domain verification
Admin	clinic lifecycle; subscriptions; plans/limits; support session; system health; announcements
Governance	audit search; export jobs; privacy requests; retention actions

16.1 Public booking-management token

- Return opaque reference plus independent secret; store only secret hash.
- Default expiry is 30 days after appointment end; clinic can shorten it.
- Token grants only appointment summary, cancellation and reschedule-request actions allowed by clinic policy.
- Token cannot expose patient history, staff details, internal notes, documents, or internal IDs.
- Rate limit attempts; rotate token after successful reschedule; revoke on clinic suspension or privacy request.

17. Patient CRM, Duplicates, Notes, and Consent

17.1 Duplicate control

- Normalize phone numbers to E.164 when country context exists; retain original display value.
- Before creation, return ranked candidates using normalized phone, email, date of birth, and name.
- Do not silently merge. Authorized staff confirms creation or merge.
- Merge preserves both identifiers, writes patient_merge_events, reassigns dependent records transactionally, and is auditable.

17.2 Note visibility

Visibility	Readers	Writers
operations	Owner, manager, permitted receptionist and doctor	Permitted clinic staff
care_team	Assigned/allowed doctors and managers if clinic policy allows	Doctors
private_doctor	Author plus explicitly authorized clinical role	Authoring doctor; correction is versioned

17.3 Consent and privacy requests

- Consent record stores purpose, versioned notice, captured_at, method, actor, withdrawal, and evidence metadata.
- Export and deletion requests are workflow records with identity verification, legal hold check, approval, completion evidence, and audit.
- Archive is not legal deletion. Approved erasure pseudonymizes or deletes eligible data and records a non-identifying completion event.
- The deployment owner must approve jurisdiction-specific retention before pilot. The product makes no automatic HIPAA/GDPR compliance claim.

18. Private Files and Public Media

Bucket	Policy
patient-private	Private; generated keys; authorization required; short-lived signed GET; no public listing.
website-public	Public published assets only; immutable versioned keys; safe media types; CDN cache allowed.
website-drafts	Private; preview authorization/token required; never indexed.

18.1 Upload pipeline

- Server validates declared MIME, extension, magic bytes, size and tenant quota.
- Object key is generated; original filename is metadata only and sanitized for display.
- Upload enters pending_scan. Downloads are blocked until clean.
- Scanner result records engine, signature version, timestamp and outcome. Failed or timed-out scans quarantine the object.
- Signed URLs default to 60 seconds and are never logged, audited verbatim, or returned to unauthorized clients.
- Document access records user, patient, document, action, request ID and time without copying content into the audit log.

18.2 Default limits

Patient document: 20 MB each; clinic public image: 8 MB; permitted patient types: PDF, JPEG, PNG; office documents disabled until separately reviewed. Images are re-encoded when practical. Executables, SVG patient uploads, archives, and HTML are rejected.

19. Website Builder and Publishing

- Three curated templates; no arbitrary HTML, JavaScript, CSS, or free-form canvas.
- Section config/content is validated against a versioned schema by section_type.
- Automatic contrast validation prevents inaccessible published color combinations.
- Preview reads draft through an authenticated or short-lived preview token and sends noindex headers.
- Publish validates all pages/assets/domains, writes an immutable snapshot, then atomically swaps live_version_id.
- Rollback creates a new version from an earlier snapshot; history is never rewritten.
- Public render reads only live_version_id and public assets.

19.1 Domain verification

domain_verifications stores requested hostname, expected DNS proof, observed status, certificate status, checked_at and failure reason. Only exact normalized hostnames map to tenants. Unknown hosts return a neutral 404; they never fall back to another clinic.

19.2 Content safety

All rich text is sanitized using an allowlist. External embeds are disabled in MVP. Links receive safe rel attributes. Medical disclaimers and privacy/cancellation pages are required before publishing; clinics own their clinical and legal claims.

20. Dashboard, Queue, Follow-ups, and Notifications

20.1 Dashboard source of truth

KPI counts and underlying lists use the same filters, status definitions, branch timezone, and permission scope. Each widget has loading, empty, error, retry and stale-data states.

20.2 Queue

- One active queue entry per appointment enforced by UNIQUE partial constraint.
- Default order is checked_in_at, with an optional audited priority override.
- Starting consultation locks the appointment version and transitions Waiting to In consultation.
- Queue views update through polling in MVP; WebSockets are deferred until measured need.

20.3 Follow-ups

Statuses: due, overdue, contacted, booked, completed, closed. Overdue is computed from due_at and clinic timezone; it is not a separately editable state. Assignment and completion are audited. Database-backed scheduled jobs create internal notifications using idempotent job keys.

20.4 Browser notifications

Optional browser push requires explicit user permission and stores endpoint credentials encrypted. Payloads contain no patient name or clinical detail; they direct the authenticated user to open the dashboard. In-app notifications remain authoritative.

21. Security and Privacy Baseline

Threat	Mandatory control
Cross-tenant access	RLS + composite FKs + tenant repositories + isolation test suite
IDOR	Tenant and object authorization for every lookup; scoped 404
Privilege escalation	Explicit permissions, protected role assignment, session rotation, audit
Session theft	TLS, Secure/HttpOnly cookie, hashing, timeout, revocation, MFA
CSRF	SameSite=Lax, strict Origin allowlist, CSRF header/token on state changes
XSS	React escaping, rich-text sanitization, CSP, no arbitrary scripts
SQL injection	Parameterized ORM/SQL; reviewed raw SQL only
Brute force	Account + IP backoff, generic responses, alerts, MFA
Malicious upload	Allowlist, magic-byte checks, scan/quarantine, private storage
Sensitive logs	Structured redaction; denylisted fields; no bodies on clinical endpoints
Admin abuse	Time-limited least-privilege support, approval, reason, expiry, audit
Supply chain	Lockfiles, dependency scanning, secret scanning, reviewed upgrades

21.1 Security headers

HSTS; CSP with nonces and restricted connect-src; X-Content-Type-Options nosniff; Referrer-Policy strict-origin-when-cross-origin; frame-ancestors none for dashboard; Permissions-Policy restricting camera, microphone and geolocation unless a feature requires them.

21.2 Encryption and secrets

- TLS in transit; managed encryption at rest for database, backups and storage.

- Application encryption key protects TOTP secrets and other approved sensitive fields.
- Secrets live in provider secret stores; never NEXT_PUBLIC, logs, fixtures, screenshots, or repository.
- Rotate production session and encryption secrets through a documented dual-key migration procedure.

22. Audit, Logging, and Observability

22.1 Audit events

Audit events are append-only and cover login/MFA/recovery, permission changes, sensitive reads, patient/document access, exports, status changes, publishing, feature limits, support access, privacy actions, and administrative configuration.

Record	Retention/default	Content rule
Security audit	2 years; configurable after legal review	Actor, action, target, outcome, request ID; no document bodies
Operational logs	30 days	No patient content, tokens, signed URLs, or request bodies
Error events	90 days	Redacted context; environment and request ID
Metrics	13 months aggregate	No direct patient identifiers

22.2 Required telemetry

- Request count, p50/p95/p99 latency, error rate, database pool saturation and slow queries.
- Booking attempts, conflicts, completion, approval delay and no-show rate without patient identifiers.
- Background job age/failures, scan backlog, backup status, storage failures and publish failures.
- Alerts for elevated 5xx, authentication attack patterns, failed backups, job backlog, storage scan outage, and tenant-isolation test failure.

22.3 Health endpoints

/health/live proves process health; /health/ready checks database and required dependencies with strict timeout; neither exposes configuration, versions, credentials, or tenant data.

23. Performance, Accessibility, and Reliability Targets

Measure	MVP launch target
Public Core Web Vitals	p75 LCP <= 2.5 s; INP <= 200 ms; CLS <= 0.1 on mobile
API list/detail	p95 <= 500 ms at agreed pilot load, excluding file transfer
Availability query	p95 <= 800 ms for 31-day range
Booking command	p95 <= 1.5 s; correct conflict result under concurrency
Dashboard	Critical data visible <= 2.5 s p75 on supported connection
Uptime	99.5% monthly MVP target, excluding announced maintenance
RPO	<= 24 hours; improve when provider plan supports point-in-time recovery
RTO	<= 8 hours using documented manual restore
Accessibility	WCAG 2.2 AA for core public and staff flows
Breakpoints	Public 320 px through desktop; staff tablet 768-1024 px and desktop

23.1 Load profile for pilot gate

Validate at least 50 concurrent staff sessions, 20 booking requests per second for a 60-second burst, one million appointments distributed across test tenants, and 100,000 patients in a large test clinic. Targets must be revisited after pilot telemetry; these are engineering test assumptions, not sales capacity promises.

23.2 Accessibility acceptance

Keyboard operation, visible focus, labels and errors, semantic landmarks, screen-reader names, 4.5:1 normal-text contrast, reduced motion, status not conveyed by color alone, and automated plus manual checks on onboarding, booking, calendar, queue, CRM, and publishing.

24. Backups, Retention, and Disaster Recovery

- Use managed encrypted PostgreSQL backups; enable point-in-time recovery when commercially available.
- Back up configuration needed to reconstruct storage policies and application environments; do not copy secrets into documentation.
- Test restoration to isolated staging before pilot and at least quarterly afterward.
- Restore test records start/end time, backup identifier, schema version, row-count checks, object-access checks, smoke tests and approver.
- Database rollback means application rollback plus backward-compatible migrations. Destructive down migrations are not the production recovery strategy.
- Storage object retention follows document metadata and legal hold. Backup deletion limitations must be disclosed in the approved retention policy.

24.1 Disaster runbook order

Declare incident; freeze risky writes if required; select verified restore point; restore database to isolated environment; validate schema, RLS and tenant counts; reconnect copied/non-production storage configuration; run smoke and isolation tests; obtain approval; switch traffic; monitor; document incident and follow-up actions.

25. CI/CD and Environment Contract

Environment	Purpose	Data rule
local	Developer execution	Synthetic fixtures only
test	Automated isolated tests	Ephemeral synthetic database
staging	Release verification and restore tests	Synthetic or irreversibly anonymized approved data
production	Clinic operations	Real data; least privilege and audited access

25.1 Pull request gate

Formatting, lint, Python and TypeScript type checks, unit/API tests, tenant/RBAC tests, migration upgrade from previous release, migration downgrade only where safe, frontend build, generated OpenAPI diff, dependency and secret scan.

25.2 Release gate

Deploy web and API to staging; run migrations as an explicit job; run Playwright critical flows, appointment concurrency, RLS isolation and health checks; approve production; deploy backward-compatible API; run migration; deploy web; verify health and smoke tests; monitor error and latency budgets.

25.3 Environment variables

```
APP_ENV=local|test|staging|production
DATABASE_URL=
DATABASE_MIGRATION_URL=
SESSION_HMAC_KEY=
FIELD_ENCRYPTION_KEYS=
COOKIE_DOMAIN=.platform.example
CSRF_ALLOWED_ORIGINS=https://app.platform.example
CORS_ORIGINS=https://app.platform.example
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SENTRY_DSN=
PUBLIC_APP_URL=https://app.platform.example
PUBLIC_WEBSITE_DOMAIN=platform.example
```

Startup must fail closed when required production variables are missing. Configuration validation must never print secret values.

26. Ordered Codex Build Plan

Codex must complete each exit gate before beginning dependent work. Parallel work is allowed only across independent modules after their shared contracts are merged.

Phase	Deliverable	Exit gate
0	ADRs, requirements map, threat model, UX flows	Decisions approved; unresolved list empty or explicitly deferred
1	Monorepo, Docker, CI, FastAPI/Next.js shells	Builds, health checks and test harness pass
2	PostgreSQL conventions, migrations, tenant context, RLS	Two-tenant isolation suite passes before clinical tables expand
3	Auth, TOTP, sessions, manual invite/reset	Revocation, CSRF, rate limits and recovery tests pass
4	RBAC and branch scopes	Every permission-role-object combination has API tests
5	Clinic onboarding, hours, doctors, services, resources	Clinic can complete and resume setup; constraints pass
6	Website templates/editor/publish/domains	Draft/live, sanitization, rollback and unknown-host tests pass
7	Availability and appointment engine	DST, overlap, idempotency and concurrent booking tests pass
8	Calendar, queue and operational dashboard	Reception critical path works on tablet and respects scope
9	Patient CRM, duplicates, notes, consent	Visibility, merge, archive and search tests pass
10	Documents and scanning	Unauthorized/expired/quarantined downloads fail
11	Follow-ups, notifications, database jobs	Due/overdue and idempotent jobs pass timezone tests
12	Plans, limits, support access, audit, privacy workflows	Limits transactional; support expires; audit complete
13	Hardening, load, accessibility, restore	All numeric targets and launch gates pass
14	Controlled pilot	Runbooks exercised; critical defects closed; owner approval

27. Codex Task Template

Use the following structure for every implementation task so the agent receives scope, constraints, and proof requirements together.

```
TASK: [Requirement IDs] short outcome
DEPENDENCIES: merged migrations/contracts/ADRs
IN SCOPE: exact modules, endpoints, UI flows
OUT OF SCOPE: explicit exclusions
DATA CONTRACT: tables, constraints, RLS, archive behavior
AUTHORIZATION: permission + tenant/object scope
API CONTRACT: request, response, errors, idempotency/versioning
AUDIT: required events and redaction
UX STATES: loading, empty, error, retry, responsive, accessibility
TESTS: unit, API, isolation, concurrency, browser
COMMANDS: exact repo lint/type/test/build commands
DONE WHEN: measurable acceptance conditions
HANDOFF: files changed, migrations, tests, risks, deviations
```

27.1 Codex completion report

- Outcome first: what now works.
- Requirement IDs completed and files changed.
- Migrations created and upgrade result.
- Commands executed with pass/fail counts.
- Security and tenant-scope evidence.
- Screens or flows visually checked where applicable.
- Remaining issues, deliberate deviations, and next safe step.

27.2 Prohibited shortcuts

No client-only permission checks; no unscoped repository helper; no broad admin bypass; no status change through generic PATCH; no public patient bucket; no plaintext token; no phone/email primary key; no silent duplicate merge; no raw rich HTML; no real patient fixtures; no skipped migration/isolation tests to make CI green.

28. Testing Contract

Suite	Mandatory coverage
Unit	Slot math, state machine, permissions, limits, normalization, redaction, schema validation
Database	Constraints, composite FKs, RLS, archives, exclusion conflicts, migration upgrade
API	Happy/failure paths, stable errors, scope, idempotency, version conflict, rate limits
Tenant	Cross-clinic read/write/delete/export/download/public token/background job/support expiry
Auth	Login, enumeration resistance, lockout, MFA, recovery, rotation, revocation, CSRF
Appointments	Hours, buffers, leave, resources, DST, notice, horizon, cancellation, reschedule, concurrency
CRM/files	Duplicates, merge, note visibility, consent, scan, size/type, signed expiry, quota
Website	Draft/live, XSS sanitization, contrast, publish failure, rollback, host resolution
Browser	Onboarding, booking, reception, queue, CRM, editor, owner security, mobile/tablet
Non-functional	Load profile, WCAG manual checks, restore, dependency outage and graceful recovery

28.1 Test-data rule

Factories generate synthetic clinics, staff, patients and documents. Names and records must be obviously fictional. Production data must never be copied into local or automated test environments. Tenant-isolation fixtures always contain at least two clinics with deliberately colliding local values.

29. Feature Definition of Done

- Requirement IDs and approved UX flow are linked.
- Database model, migration, constraints, RLS and archive behavior exist.
- API schema, permissions, object scope, stable errors and audit events exist.
- Happy path, validation, forbidden, cross-tenant, conflict and failure tests pass.
- Frontend includes loading, empty, error, retry, success and destructive confirmation states.
- Keyboard, screen-reader name, focus and contrast behavior are verified.
- Tablet/mobile behavior is verified for the applicable surface.
- Logs and Sentry context are redacted; no secret reaches the browser.
- Feature limit is enforced transactionally where applicable.
- OpenAPI and generated frontend types are current.
- Runbook and operational metrics are updated where required.
- All relevant CI checks pass and deviations are documented.

30. Production Launch Gate

- Automated and manual cross-tenant tests show no data exposure.
- Composite tenant foreign keys and mandatory RLS are enabled and forced on sensitive tables.
- Appointment exclusion constraints and concurrent-booking tests pass.
- Owner/platform-admin TOTP, recovery, session rotation and revocation work.
- CSRF, CORS, CSP, rate limiting and secure-cookie checks pass in production topology.
- Private files cannot be read before scan, outside scope, or through expired URLs.
- Draft/live publishing, rollback, sanitization and custom-domain isolation pass.
- Backup restoration passes in isolated staging within RTO.
- Core performance and WCAG 2.2 AA targets pass.
- Plans and limits are enforced by backend transactions.
- All administrative/support actions are auditable and support sessions expire.
- Secrets are absent from source, client bundles, logs and error events.
- Retention, consent, privacy-request and incident procedures are approved for the deployment jurisdiction.
- Monitoring alerts, on-call ownership, rollback and incident runbooks are tested.
- Pilot clinic owners approve the operational workflow.

31. Resolved Gaps and Decision Register

Earlier gap	Resolution in v2.0
Railway or Render ambiguity	Railway selected; Render is contingency requiring ADR.
RLS optional	RLS mandatory and forced on sensitive clinic tables.
Same-tenant FK weakness	Composite tenant foreign keys required.
Booking race not executable	GiST exclusion constraint + transactional recheck + idempotency.
Missing domain tables	Hours, holidays, resources, questions, tags, consent, tokens, scans, domains, privacy and jobs added.
Incomplete API inventory	All major public, clinic, admin and governance endpoint groups defined.
Generic appointment PATCH	Dedicated transition/reschedule commands with version checks.
No-message auth contradiction	Manual one-time setup/reset links with explicit offline delivery.
MFA unspecified	TOTP and hashed recovery codes selected.
Cross-domain session unclear	Exact app/API/custom-domain cookie, CORS and CSRF boundaries set.
Public booking management unclear	Hashed limited-scope token with expiry, rate limits and rotation.
File scanning incomplete	Scan/quarantine pipeline, metadata and download gate defined.
Privacy generic	Consent, privacy-request, retention approval and no-compliance-claim rules added.
Performance vague	Numeric latency, Web Vitals, uptime, RPO/RTO and load assumptions added.
Codex usage unclear	Instruction precedence, stop conditions, ordered phases, task and handoff templates added.
PDF clipping	Tables restructured for portrait width and repeated headers.

32. Final Acceptance Statement

This specification is suitable as the implementation baseline for Codex and an engineering team. It converts the product roadmap into ordered technical contracts, closes the identified architectural and workflow gaps, and makes security, tenant isolation, appointment integrity, verification, and delivery evidence part of every feature rather than post-launch work.

Build-ready does not mean legally certified. Before processing real patient data, the deployment owner must complete jurisdiction-specific legal, privacy, retention, contractual, and operational review.

Final principle	Meaning
Simple surface	Doctors and receptionists can act without training or technical knowledge.
Disciplined core	Tenant isolation, database invariants, audit, privacy and tests are never optional.
Measured release	The MVP launches only when objective acceptance gates pass.
Codex-ready execution	Every task has scope, contracts, checks, evidence and a safe stopping rule.

END OF SPECIFICATION